

K8S_Task03 namespace & MetalLB

1. Create blue and red namespaces

kubectl create namespace blue

kubectl create namespace red

kubectl get ns

```
root@k8s-control:~# kubectl create namespace blue
namespace/blue created
root@k8s-control:~# kubectl create namespace red
namespace/red created
root@k8s-control:~# kubectl get ns
NAME                STATUS    AGE
blue                 Active    9s
default              Active    14d
kube-node-lease      Active    14d
kube-public           Active    14d
kube-system           Active    14d
red                  Active    5s
root@k8s-control:~#
```

2. Create deployment & service on blue namespace for Nexus

vim nexus-deployment.yaml

kubectl apply -f nexus-deployment.yaml

```
root@k8s-control:~# kubectl get svc -n blue
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
nexus-service       ClusterIP   10.102.242.185 <none>       8081/TCP    3m59s
root@k8s-control:~# kubectl get all -n blue
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
service/nexus-service ClusterIP   10.102.242.185 <none>       8081/TCP    4m4s
root@k8s-control:~#
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nexus
  namespace: blue
  labels:
    app: nexus
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nexus
  template:
    metadata:
      labels:
        app: nexus
    spec:
      containers:
        - name: nexus
          image: sonatype/nexus3
      ports:
        - containerPort: 8081
```

```
apiVersion: v1
kind: Service
metadata:
  name: nexus
  namespace: blue
spec:
  type: ClusterIP
  selector:
    app: nexus
  ports:
    - protocol: TCP
      port: 8081
      targetPort: 8081
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nexus
  namespace: blue
  labels:
    app: nexus
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nexus
  template:
    metadata:
      labels:
        app: nexus
    spec:
      containers:
        - name: nexus
          image: sonatype/nexus3
          ports:
            - containerPort: 8081
---
apiVersion: v1
kind: Service
metadata:
  name: nexus
  namespace: blue
spec:
  type: ClusterIP
  selector:
    app: nexus
  ports:
    - protocol: TCP
      port: 8081
      targetPort: 8081

```

3. Create nginx-proxy-config

```
# vim nginx-proxy-config.yaml
```

```
# kubectl apply -f nginx-proxy-config.yaml
```

```

root@k8s-control:~# vim nginx-proxy-config.yaml
root@k8s-control:~# kubectl apply -f nginx-proxy-config.yaml
configmap/nginx-proxy-config created
root@k8s-control:~#

```

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-proxy-config
  namespace: red
data:
  default.conf: |
    server {
      listen 80;

      location / {
        proxy_pass http://nexus.blue.svc.cluster.local:8081;

        proxy_set_header Host nexus.blue.svc.cluster.local;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
      }
    }

```

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-proxy-config
  namespace: red
data:
  default.conf: |
    server {
      listen 80;

      location / {
        proxy_pass http://nexus.blue.svc.cluster.local:8081;

        proxy_set_header Host nexus.blue.svc.cluster.local;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
      }
    }

```

4. Create deployment & service on red namespace for Nginx

```

# vim nginx-deployment.yaml
# kubectl apply -f nginx-deployment.yaml
# kubectl get all -n red

```

```

root@k8s-control:~# vim nginx-deployment.yaml
root@k8s-control:~# kubectl apply -f nginx-deployment.yaml
service/nginx-service created
root@k8s-control:~# kubectl get all -n red

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/nginx-service	NodePort	10.110.191.24	<none>	80:30150/TCP	7s

```

root@k8s-control:~#

```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: red
  labels:
    app: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
          volumeMounts:
            - name: nginx-config-volume
              mountPath: /etc/nginx/conf.d/default.conf
              subPath: default.conf
      volumes:
        - name: nginx-config-volume
          configMap:
            name: nginx-proxy-config
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
  namespace: red
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30150
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: red
  labels:
    app: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
          volumeMounts:
            - name: nginx-config-volume
              mountPath: /etc/nginx/conf.d/default.conf
              subPath: default.conf
      volumes:
        - name: nginx-config-volume
          configMap:
            name: nginx-proxy-config
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
  namespace: red
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30150 # istəşən konkret port təyin edə bilərsən (30000-32767 aralığı), yoxsa avtomatik seçiləcək

```

5. Check

kubectl get pods -n blue -o wide

kubectl get pods -n red -o wide

```

root@k8s-control:~# kubectl get pod -n blue -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE           NOMINATED NODE   READINESS GATES
nexus-6ff56d8754-cnxx6             1/1    Running   0           18m   192.168.79.65   k8s-worker01   <none>           <none>
root@k8s-control:~# kubectl get pod -n red -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE           NOMINATED NODE   READINESS GATES
nginx-84cd8c8769-csd6             1/1    Running   1 (7m35s ago)   8m34s   192.168.69.193   k8s-worker02   <none>           <none>
root@k8s-control:~# █

```

6. Go to web <http://192.168.153.11:30150/>

The screenshot shows a web browser window with the address bar displaying "192.168.153.11:30150/#/login". The page title is "Login - Sonatype Nexus Repository". The Sonatype Nexus Repository logo is at the top left. The main content area is a light gray box with a white login card on the right. The card has the Sonatype logo, the text "Welcome", and "Login to continue". A blue information box states: "To log in for the first time, use the generated admin password located at: /nexus-data/admin.password". Below this are input fields for "Username" and "Password", and a "Log in" button.

sonatype nexus repository
Community Edition

Welcome

Login to continue

To log in for the first time, use the generated admin password located at: /nexus-data/admin.password

Username

Password

Log in

The screenshot shows the dashboard of the Sonatype Nexus Repository. The address bar displays "192.168.153.11:30150/#/browse/welcome". The page title is "Dashboard - Sonatype Nexus Repository". The Sonatype Nexus Repository logo is at the top left. The main content area is a light gray box with a white dashboard card on the right. The card has the Sonatype logo, the text "sonatype nexus repository", and "Community Edition". Below the logo are three cards: "Browse" (Browse my repositories), "Connect" (Connect to a repository), and "Release Notes" (See what's new).

sonatype nexus repository
Community Edition

Browse
Browse my repositories

Connect
Connect to a repository

Release Notes
See what's new

The screenshot shows the "Usage Center" of the Sonatype Nexus Repository. The address bar displays "192.168.153.11:30150/#/browse/welcome". The page title is "Dashboard - Sonatype Nexus Repository". The Sonatype Nexus Repository logo is at the top left. The main content area is a light gray box with a white usage center card on the right. The card has the Sonatype logo, the text "Usage Center", and "Usage below limits". Below the logo is a search bar. The main content area is a light gray box with a white usage center card on the right. The card has the Sonatype logo, the text "Usage Center", and "Usage below limits". Below the logo is a search bar. The main content area is a light gray box with a white usage center card on the right. The card has the Sonatype logo, the text "Usage Center", and "Usage below limits". Below the logo is a search bar.

sonatype nexus repository
Community Edition

Usage Center
Usage below limits

Monitor this instance's usage to ensure your deployment is appropriate for your needs. Learn more about the usage center.

Usage Metrics Overview

Total Components	Requests Per Day	Requests Per Month
0 Current 40,000 Usage Limit	0 Last 24 hours 100,000 Usage Limit	0 Total requests in August
0 Highest Recorded Count (30 days)	0 Highest Recorded Count (30 days)	0 Average requests (12 months)
		0 Highest recorded count (12 months)

System Health Check
View system status checks

Cleanup Policies
Review component removal policies

Browse
Browse my repositories

7. Create test1 service

vim test1-svc.yaml

kubectl apply -f test1-svc.yaml

```
root@k8s-control:~# vim test1-svc.yaml
root@k8s-control:~# kubectl apply -f test1-svc.yaml
service/test1 created
root@k8s-control:~#
```

apiVersion: v1

kind: Service

metadata:

name: test1

namespace: red

spec:

type: LoadBalancer

loadBalancerIP: 192.168.153.10

selector:

app: nginx

ports:

- protocol: TCP

port: 80

targetPort: 80

```
apiVersion: v1
kind: Service
metadata:
  name: test1
  namespace: red
spec:
  type: LoadBalancer
  loadBalancerIP: 192.168.153.10
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```


8. Create MetalLB

kubectl apply -f

<https://raw.githubusercontent.com/metallb/metallb/v0.16.1/config/manifests/metallb-native.yaml>

kubectl get pods -n metallb-system

```
root@k8s-control:~# kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.16.1/config/manifests/metallb-native.yaml
namespace/metallb-system created
customresourcedefinition.apiextensions.k8s.io/bfdprofiles.metallb.io created
customresourcedefinition.apiextensions.k8s.io/bgpadvertisements.metallb.io created
customresourcedefinition.apiextensions.k8s.io/bgppeers.metallb.io created
customresourcedefinition.apiextensions.k8s.io/communities.metallb.io created
customresourcedefinition.apiextensions.k8s.io/configurationstates.metallb.io created
customresourcedefinition.apiextensions.k8s.io/ipaddresspools.metallb.io created
customresourcedefinition.apiextensions.k8s.io/l2advertisements.metallb.io created
customresourcedefinition.apiextensions.k8s.io/servicebgpstatuses.metallb.io created
customresourcedefinition.apiextensions.k8s.io/servicel2statuses.metallb.io created
serviceaccount/controller created
serviceaccount/speaker created
role.rbac.authorization.k8s.io/controller created
role.rbac.authorization.k8s.io/pod-lister created
clusterrole.rbac.authorization.k8s.io/metallb-system:controller created
clusterrole.rbac.authorization.k8s.io/metallb-system:speaker created
rolebinding.rbac.authorization.k8s.io/controller created
rolebinding.rbac.authorization.k8s.io/pod-lister created
clusterrolebinding.rbac.authorization.k8s.io/metallb-system:controller created
clusterrolebinding.rbac.authorization.k8s.io/metallb-system:speaker created
configmap/metallb-exclude12 created
secret/metallb-webhook-cert created
service/metallb-webhook-service created
deployment.apps/controller created
daemonset.apps/speaker created
validatingwebhookconfiguration.admissionregistration.k8s.io/metallb-webhook-configuration created
root@k8s-control:~#
```

```
root@k8s-control:~# kubectl get pods -n metallb-system
NAME                                READY   STATUS              RESTARTS   AGE
controller-56cbc7c68d-jbx7b        0/1     Running             0           101s
speaker-74pfq                      0/1     ContainerCreating   0           101s
speaker-8jlbx                      0/1     ContainerCreating   0           101s
speaker-qpb2c                      0/1     ContainerCreating   0           101s
root@k8s-control:~#
```

9. Create ip-pool

vim ip-pool.yaml

kubectl apply -f ip-pool.yaml

apiVersion: metallb.io/v1beta1

kind: IPAddressPool

metadata:

name: test-pool

namespace: metallb-system

spec:

addresses:

- 192.168.153.10-192.168.153.20

autoAssign: false

apiVersion: metallb.io/v1beta1

kind: L2Advertisement

metadata:

name: l2-advert

namespace: metallb-system

spec:

ipAddressPools:

- test-pool

```

apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: test-pool
  namespace: metallb-system
spec:
  addresses:
    - 192.168.153.10-192.168.153.20
  autoAssign: false
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: l2-advert
  namespace: metallb-system
spec:
  ipAddressPools:
    - test-pool

```

```

root@k8s-control:~# vim ip-pool.yaml
root@k8s-control:~# kubectl apply -f ip-pool.yaml
ipaddresspool.metallb.io/test-pool created
l2advertisement.metallb.io/l2-advert created
root@k8s-control:~#

```

10. Go to web <http://192.168.153.10/>

